



南開大學
Nankai University

计算机学院
并行程序设计实验报告

CPU 架构编程

姓名：王一诺

学号：2312331

专业：计算机科学与技术

2025 年 3 月 25 日

目录

1	实验平台配置	2
2	实验一：$n \times n$ 矩阵与向量内积	2
2.1	算法设计	2
2.1.1	平凡算法设计思路	2
2.1.2	Cache 优化算法设计思路	2
2.2	编程实现	2
2.3	性能测试	3
2.3.1	测试结果（不同规模对比）	3
2.3.2	关键观察与尝试解释	3
2.4	Profiling（VTune 分析）	4
2.4.1	缓存命中率分析	4
2.5	结果分析	4
3	实验二：n 个数求和	5
3.1	算法设计	5
3.1.1	平凡算法设计思路	5
3.1.2	指令级并行算法设计思路	5
3.2	编程实现	5
3.3	性能测试	6
3.3.1	测试结果（不同规模对比）	6
3.3.2	关键观察与尝试解释	6
3.4	Profiling（VTune 分析）	7
3.4.1	CPI 分析	7
3.5	结果分析	7
4	实验总结和思考	7
4.1	对比两个实验的异同	7
4.2	总结	8
4.3	进一步优化思考	8
5	链接	8

1 实验平台配置

配置 \ 需求	编译	程序开发	性能测试
X86	TDM-GCC	Code::Blocks	VTune
CPU 型号	CPU 主频	L1/L2/L3 Cash	
Intel Core i9-14900HX	2200MHz	2.1MB/32MB/36MB	

表 1: 实验环境

2 实验一：n×n 矩阵与向量内积

2.1 算法设计

平凡算法按列访问矩阵元素，但 C/C++ 中矩阵是行主存储，导致跨行跳跃访问，破坏了空间局部性。每次加载的 Cache Line（如 64B 含 8 个 double 元素）仅用 1 个元素后失效，剩余 7 个浪费，频繁触发 Cache Miss。优化算法改为按行访问，每一步外层循环为所有列累加一个行元素与向量的乘积，使得每次加载的 Cache Line 被完全利用，显著提升缓存命中率，充分发挥 Cache 容量优势。

2.1.1 平凡算法设计思路

核心逻辑：逐列遍历矩阵元素，每一步外层循环计算一列与向量的内积。

访存模式：矩阵按列访问（C/C++ 中矩阵为行主存储），导致空间局部性差，频繁触发 Cache Miss。

2.1.2 Cache 优化算法设计思路

核心逻辑：逐行遍历矩阵元素，每一步外层循环为所有列的部分内积累加一个乘法结果。

访存模式：矩阵按行访问，与行主存储匹配，充分利用空间局部性，减少 Cache Miss。

2.2 编程实现

高精度计时测试函数

```
1 void measure_time(void (*func)(), const char* name) {
2     LARGE_INTEGER head, tail, freq;
3     QueryPerformanceFrequency(&freq);
4     QueryPerformanceCounter(&head);
5     for (int i = 0; i < 100; i++) { // 重复执行100次以减小误差
6         func();
7     }
8     QueryPerformanceCounter(&tail);
9     double time = (tail.QuadPart - head.QuadPart) * 1000.0 / freq.QuadPart;
10    cout << name << "总用时" << time << " ms "; << "平均时间: " << time / 100 << " ms"
11    << endl;
12 }
```

平凡算法：逐列访问

```
1 void naive_column_dot() {
2     for (int i = 0; i < N; i++) {
3         col_sum[i] = 0.0;
4         for (int j = 0; j < N; j++) {
5             col_sum[i] += matrix[j][i] * vector[j]; //列主序访问
6         }
7     }
8 }
```

Cache 优化算法：逐行访问

```
1 void cache_optimized_dot() {
2     for (int i = 0; i < N; i++) {
3         col_sum[i] = 0.0;
4     }
5     for (int j = 0; j < N; j++) {
6         double v = vector[j];
7         for (int i = 0; i < N; i++) {
8             col_sum[i] += matrix[j][i] * v; //行主序访问
9         }
10    }
11 }
```

2.3 性能测试

2.3.1 测试结果（不同规模对比）

矩阵规模 (N)	平凡算法平均时间 (ms)	Cache 优化算法平均时间 (ms)	性能提升倍数
64	0.004146	0.002851	1.454x
128	0.020798	0.014062	1.479x
256	0.081279	0.049874	1.630x
512	0.383206	0.131796	2.908x
1024	5.17423	0.425339	12.165x
2048	16.7796	2.47687	6.775x
4096	87.6138	9.34169	9.379x
8192	503.446	36.9761	13.615x
10000	520.511	50.6183	10.283x

表 2: 测试结果

2.3.2 关键观察与尝试解释

根据实验数据与三级缓存配置（L1:2.1MB，L2:32MB，L3:36MB），结合矩阵内存占用（ $8N^2$ 字节），可以对实验结果做出一定分析和尝试解释。

当矩阵规模较小时（如 $N=64$ ，内存占用 32KB），数据完全容纳于 L1 Cache（2.1MB）。此时，平凡算法（逐列访问）与优化算法（逐行访问）的时间差距较小。然而，平凡算法的跨列访问模式导致

Cache Line 利用率较低，引发 L1 Miss 率升高。尽管数据完全在 L1 中，这种低效的局部性仍会导致流水线停滞。优化算法通过连续的行主序访问，使 Cache Line 高效利用，减少 Miss 次数，性能差距逐步扩大。

当矩阵规模增至 $N=512$ （内存占用 2MB）时，数据仍可完全保留在 L1 Cache 中，但平凡算法的跨列访问开始引发 L1 Cache 冲突，性能差距拉大至 2.9 倍。当 $N=1024$ （内存占用 8MB）时，数据进入 L2 Cache（32MB），平凡算法的访存模式彻底暴露缺陷：跨行跳跃访问导致 L2 Cache Line 无法有效复用，L2 Miss 率飙升，每次 Miss 需从 L3 加载数据（延迟约 40ns），CPI 增大。而优化算法凭借行主序访问的连续性，在 L2 中保持接近 100% 命中率，数据无需下沉至 L3，CPI 相对稳定，性能差距扩大至 12.2 倍。

当矩阵规模超过 L3 Cache 容量（ $N=4096$ 时内存占用 128MB），内存访问延迟占主导。平凡算法的性能进一步崩塌，其跨列访问导致内存访问高度随机，引发页表跳跃，有效带宽利用率较低。且此时，平凡算法的 L3 Miss 率较高，会增大访存开销。优化算法则通过内存访问连续性维持优势。

2.4 Profiling（VTune 分析）

2.4.1 缓存命中率分析

		N=256		N=1024		N=4096	
		<i>Ordinary</i>	<i>Cache</i>	<i>Ordinary</i>	<i>Cache</i>	<i>Ordinary</i>	<i>Cache</i>
指标	<i>L1HIT</i>	60,000,180	58,000,174	974,002,922	986,002,958	14,904,044,712	14,674,044,022
	<i>L1MIS</i>	1,600,024	800,012	89,201,338	4,800,072	1,652,824,792	88,001,320
	<i>L1HitRate</i>	97.4%	98.6%	91.6%	99.5%	90.0%	99.4%
	<i>L2HIT</i>	1,600,024	0	2,400,036	5,200,078	102,801,542	86,401,296
	<i>L2MIS</i>	0	0	87,018,270	0	1,548,725,164	2,000,420
	<i>L2HitRate</i>	100%	0%	2.7%	100%	6.2%	97.7%
	<i>L3HIT</i>	0	0	82,817,388	200,042	1,289,470,732	0
	<i>L3MIS</i>	0	0	600,252	0	290,822,094	2,000,840
	<i>L3HitRate</i>	0%	0%	99.3%	100%	81.6%	0%

表 3: 缓存命中率分析

2.5 结果分析

为了印证之前的分析和解释，采集了数据规模在 256，1024 和 4096 的两组实验，通过 VTune 分析其各级缓存的访存次数和命中率，如表??所示。

在小规模阶段（ $N=256$ ，矩阵占用 512KB），数据完全容纳于 L1 Cache（2.1MB），但平凡算法的逐列访问导致 Cache Line 利用率低下，引发 L1 Miss 率 3.7%（命中率 96.3%），触发 160 万次 L2 访问（全部命中）。优化算法通过行主序访问使 L1 命中率提升至 98.6%，L2 访问次数为 0，此时 L2 的高效命中掩盖了平凡算法的局部性缺陷，两者耗时差距仅 1.6 倍。

当规模增至 $N=1024$ （矩阵 8MB），数据进入 L2 Cache（32MB），平凡算法的跨列访问彻底破坏空间局部性，L1 Miss 率升至 8.4%（命中率 91.6%），触发 8920 万次 L2 访问，但 L2 命中率仅 2.7%（240 万次命中），剩余 8680 万次 L2 Miss 下沉至 L3，引发 8340 万次 L3 访问（命中率 95.2%）。优化算法凭借行主序连续性，L2 命中率保持 100%，L3 访问仅 20 万次，耗时差距拉大至 12.2 倍，印证了 L2 局部性崩塌对性能的毁灭性影响。

在超大规模阶段（ $N=4096$ ，矩阵 128MB），数据远超 L3 容量（36MB），平凡算法 L3 Miss 率达 18.4%（2.9 亿次 Miss），触发 2.9 亿次内存访问，耗时显著增大；优化算法通过连续内存访问高效利

用 cache, 利用 L1 和 L2 的高命中处理大多数数据, 虽然 L3 高 Miss 有部分进入内存, 但量小且优化算法能通过内存访问连续性维持优势, 性能差距来到 9.4 倍。

3 实验二：n 个数求和

3.1 算法设计

对于该问题, 平凡算法顺序累加 ($\text{sum} += a[i]$) 存在强数据依赖 (每步依赖前一步的 sum), 导致流水线频繁停滞, IPC (指令/周期) 被限制。多链路方法 (如两路链式累加) 将累加拆分为多条独立计算链 ($\text{sum1} += a[i]$ 和 $\text{sum2} += a[i+1]$), 消除依赖后, 在一个循环内同时用着多个临时变量对多个不同的位置进行累加, 达到多个位置并行累加的效果, IPC 提升。而递归分治通过分治策略进一步减少跨步访问, 但受限于函数调用开销, 性能弱于循环展开。

3.1.1 平凡算法设计思路

核心逻辑: 顺序累加所有元素, 单链式依赖。

性能瓶颈: 指令级并行性差, 无法充分利用超标量流水线。

3.1.2 指令级并行算法设计思路

两路链式累加: 将累加拆分为两条独立链, 减少数据依赖。

递归分治算法: 两两相加后递归合并, 利用分治减少依赖。

二重循环分阶段累加: 非递归实现, 通过循环分阶段合并中间结果。

3.2 编程实现

平凡算法: 链式累加

```
1 double naive_sum(double* data_array, int n) {  
2     double sum = 0.0;  
3     for (int i = 0; i < n; i++) {  
4         sum += data_array[i];  
5     }  
6     return sum;  
7 }
```

两路链式累加

```
1 double unrolled_sum(double* data_array, int n) {  
2     double sum1 = 0.0, sum2 = 0.0;  
3     for (int i = 0; i < n; i += 2) {  
4         sum1 += data_array[i];  
5         sum2 += data_array[i + 1];  
6     }  
7     return sum1 + sum2;  
8 }
```

递归分治算法

```
1 void recursive_sum(double* a, int n) {  
2     if (n == 1) return;  
3     for (int i = 0; i < n / 2; i++) {  
4         a[i] += a[n - i - 1];  
5     }  
6     recursive_sum(a, n / 2);  
7 }
```

二重循环分阶段累加

```
1 double iterative_sum(double* a, int n) {  
2     int m = n;  
3     while (m > 1) {  
4         for (int i = 0; i < m / 2; i++) {  
5             a[i] = a[i * 2] + a[i * 2 + 1];  
6         }  
7         m /= 2;  
8     }  
9     return a[0];  
10 }
```

3.3 性能测试

3.3.1 测试结果（不同规模对比）

n	平凡算法 (ms)	两路链式 (ms)	递归分治 (ms)	二重循环 (ms)
9	0.002315	0.001176	0.001728	0.001981
11	0.01142	0.00462	0.00617	0.00663
13	0.0337	0.0158	0.02008	0.02154
15	0.08932	0.04832	0.05855	0.06259
17	0.41218	0.19498	0.32907	0.4541
18	0.98426	0.5837	0.68102	0.52231
19	1.94406	1.04768	1.08826	0.70162
21	3.26595	1.8186	2.16697	2.82545
23	13.1043	6.96917	10.6366	12.0525
24	26.7308	14.5883	18.5498	22.6445
25	52.1121	29.1828	39.7539	48.8386

表 4: 平均用时

3.3.2 关键观察与尝试解释

两路链式累加通过指令级并行性和连续内存访问，在中小规模下保持绝对优势；递归分治因分治策略的渐进局部性优化，在极端规模下表现稳健；二重循环通过分阶段内存压缩，在特定规模区间具备优势。链式累加则因强数据依赖和随机访问模式全程垫底。

3.4 Profiling (VTune 分析)

3.4.1 CPI 分析

<i>naive_{sum}</i>		规模			
		<i>n</i> = 15	<i>n</i> = 18	<i>n</i> = 21	<i>n</i> = 24
指标	<i>CPUTime</i>	1.653ms	4.960ms	9.921ms	297.619ms
	<i>Clockticks</i>	2,000,003	24,000,036	22,000,033	1,390,002,085
	<i>InstructionsRetired</i>	4,000,006	36,000,054	38,000,057	2,350,003,525
	<i>CPIRate</i>	0.500	0.667	0.579	0.591

表 5: 平凡算法 CPI

<i>unrolled_{sum}</i>		规模			
		<i>n</i> = 15	<i>n</i> = 18	<i>n</i> = 21	<i>n</i> = 24
指标	<i>CPUTime</i>	0usec	4.134ms	33.896ms	152.117ms
	<i>Clockticks</i>	2,000,003	10,000,015	90,000,135	736,001,104
	<i>InstructionsRetired</i>	4,000,006	30,000,045	256,000,384	2,014,003,021
	<i>CPIRate</i>	0.500	0.333	0.352	0.365

表 6: 两路链式 CPI

3.5 结果分析

从 VTune 数据可见，两路链式累加算法在不同规模下均显著优于平凡算法，其核心优势源于指令级并行性优化与流水线效率提升。可从以下两方面解析：

1. 指令级并行性提升：平凡算法的单链式累加 ($\text{sum} += a[i]$) 存在强数据依赖，每条加法指令需等待前一条结果，导致流水线频繁空闲。两路链式将累加拆分为独立的两条链，消除依赖后，超标量流水线可同时执行两组加法指令，IPC 提升。从指令数对比看， $n=24$ 时两路链式仅需 20.1 亿条指令，少于平凡算法的 23.5 亿条，表明其通过减少循环指令数进一步优化了指令效率。

2. 流水线停滞减少：平凡算法的强依赖导致流水线停滞，以 $n=24$ 为例， $\text{CPI}=0.591$ 意味着每指令平均消耗 0.59 周期，而两路链式 $\text{CPI}=0.365$ ，效率提升 38%。这一差距在小规模 ($n=15$) 时本应显现，但因指令数少，时间差异不显著；当规模扩大至 $n=18$ 时，两路链式 CPI 降至 0.333，耗时进一步缩小。随着规模进一步增大 ($n=24$)，两路链式可能是通过减少分支预测失败率和缓存局部性优化，进一步巩固了优势。

4 实验总结和思考

4.1 对比两个实验的异同

对比维度	矩阵内积实验	<i>n</i> 数求和实验
优化目标	提升 Cache 局部性	充分发挥 CPU 超标量优化
关键优化手段	行主序访问	超标量并行、分治策略
性能瓶颈	Cache Miss	数据依赖、分支预测失败

表 7: 两实验对比

4.2 总结

Cache 优化优先级：矩阵计算中，改善访存模式（如行主序访问）的收益很高，尤其当数据规模超过缓存容量时。

指令级并行潜力：在计算密集型任务中，通过减少数据依赖（如超标量并行、循环展开）可显著提升 IPC，但需减少函数调用开销。

规模敏感性：

矩阵实验：性能拐点在数据规模超过 L2 Cache 时（ $N=256$ ）。

求和实验：递归算法在 $n>20$ 时性能急剧下降，受栈深度等因素影响。

工具验证必要性：善用 VTune 等性能分析工具精准定位瓶颈（如 Cache Miss、IPC），避免盲目优化。

4.3 进一步优化思考

矩阵内积：将矩阵划分为与 Cache 大小匹配的子块，可能进一步提升效率。

n 数求和：展开为 4 路或 8 路，可能进一步减少依赖。

5 链接

Gitee 链接 [Gitee](#)